



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

解释器模式（Interpreter）

一、意图（Intent）

在嵌入式 C 里，解释器模式要解决的不是“收到一个命令就分发出去”，而是：

当输入本身已经是一门小语言时，把它按约定语法拆开、理解，并得到可执行含义。

它控制的是：

- 输入到底由哪些语法单元组成
- 各个单元怎样组合成完整语义
- 解释时依赖哪些上下文状态

所以解释器模式真正面对的是**语法**，而不是单纯的“命令号分派”。

二、本质 / 动机（Motivation）

很多嵌入式系统最后都会长出一门“小语言”，只是规模大小不同。

典型来源有：

- 串口 shell
- AT 命令
- 调试命令行
- 启动脚本
- 测试脚本

- 规则配置
- 简化表达式

如果把这些输入全部手写成一堆 `strcmp()`、`if/else`、位置偏移判断，短期能跑，长期通常会出问题。

1. 语法规则会散掉

最开始也许只有几条命令，后面一旦出现：

- 子命令
- 可选参数
- 变量
- 条件语句
- 多条命令串接

零散的字符串判断会迅速失控。

2. 输入结构与执行逻辑混在一起

系统既要负责判断“这串字符是什么意思”，又要负责“实际执行什么动作”。这样一来，解析器、状态机、执行器会互相缠在一起。

3. 扩展成本越来越高

每新增一个新语法，都要改大量已有判断。

很容易出现：

- 旧命令被新规则误伤
- 某些路径漏了参数检查
- 错误提示不一致

4. 很多开源项目和厂家工具链默认就把它当“小语言”处理

无论是 bootloader 命令行、RTOS shell，还是模组 AT 命令，真正稳定的实现通常都不是“直接比较整行字符串”，而是先做：

- 词法拆分
- 参数解释
- 子命令树匹配
- 语义执行

这就是解释器模式在嵌入式中的典型落点。

三、适用性（Applicability）

解释器模式适合这些条件：

- **输入有语法**：不是一个命令码，而是一段有规则的文本或 token
- **规则相对稳定**：语法结构不会每天大改

- **要反复解释**：系统长期处理同一套语法
- **命令族逐渐增多**：存在主命令、子命令、参数组合
- **需要更好的可维护性**：希望“语法”和“执行”分层

一句话判断：

如果输入已经像一门小语言，而不是一个简单枚举值，就应该用解释器的视角去设计。

四、结构（Structure）

在嵌入式 C 场景里，可以把解释过程看成：

```
输入流
  |
  v
词法拆分 / 参数切分
  |
  v
语法节点匹配
  |
  v
语义解释
  |
  v
执行动作
```

若把角色写完整，常见关系是：

```
Input -> Token/Command Tree -> Expression -> Context -> Executor
```

这里要注意：

- 解释器并不一定非要构建复杂 AST
- 很多 MCU 项目更常见的是“命令树 + 参数解释 + 上下文执行”
- 只要输入被按一套语法理解，这仍然是解释器模式的范畴

五、参与者（Participants）

1. AbstractExpression（统一解释接口）

它定义“这个语法单元怎样解释”。

在 C 里，这通常表现为：

- 一组统一解释函数
- 命令节点回调接口
- 子命令解析器接口

重点不在面向对象外形，而在：

不同语法单元都按统一协议参与解释。

2. TerminalExpression（终结符）

它表示最小语法单位，例如：

- 一个关键字
- 一个数字
- 一个变量名
- 一个固定选项

它负责把最小单元解释成可以使用的值。

3. NonterminalExpression（非终结符）

它表示由多个语法单元组合成的结构，例如：

- 一条完整命令
- 一个带参数的子命令
- 一段条件表达式
- 一条脚本语句

它负责组合子规则，得到更完整的语义。

4. Context（上下文）

在嵌入式系统里，这个角色通常非常重要。

它往往包含：

- 当前输入位置
- 参数表
- 环境变量
- 当前设备状态
- 查找表
- 命令执行环境

解释器不是只靠字符串本身就能工作，往往还要依赖这些上下文。

5. Client（发起解释者）

Client 负责把输入交给解释系统，并在解释完成后触发对应动作。

它通常是：

- shell 主循环
 - CLI 收发线程
 - bootloader 命令入口
 - AT 命令入口
-

六、协作（Collaborations）

典型执行路径如下：

1. 系统收到一段输入
2. 先按规则切分出 token 或参数
3. 再根据语法结构找到对应节点或表达式
4. 每个节点在上下文中解释自己的含义
5. 多个子单元组合后得到完整语义
6. 最终交给执行层完成动作

这里最重要的点是：

系统不是“看见一串字符就直接调函数”，而是先按语法理解，再决定动作。

这就是解释器模式和普通命令分派的本质区别。

七、效果（Consequences）

收益

1. 语法结构更清楚

命令字、参数、子命令、变量、条件这些角色可以分层表达，而不是全堆在一堆字符串比较里。

2. 更适合扩展小语言

随着命令族变多、参数组合变复杂，解释器模式通常比手写分支更稳。

3. 语法与执行可以分层

“输入怎样理解”和“理解后做什么”不必完全混在一起。

4. 错误处理更一致

非法命令、参数缺失、语法错误、上下文不满足等问题，更容易统一处理。

代价

1. 文法一大就会变复杂

如果语法已经接近完整脚本语言，解释器层会迅速膨胀。

2. 增加了解析和上下文管理成本

在资源很紧的小 MCU 上，要控制实现规模。

3. 不适合特别简单的命令码场景

如果输入只是几个固定枚举值，没有语法结构，那么没必要把它做成解释器。

八、实现要点 (Implementation)

1. 先区分“命令分派”与“语法解释”

如果只是根据一个 ID 或一个函数表查找处理器，那更像普通分派；
如果要理解输入结构，那才是解释器模式的真正应用点。

2. 优先做清晰的 token/参数层

不要一上来就在业务代码里到处直接切字符串。
先把词法切分、参数提取、合法性检查层整理好，会让后续扩展轻松很多。

3. 上下文要可控

解释器若要访问设备状态、变量区、权限状态，必须明确这些状态由谁持有、何时有效。

4. 错误路径要单独设计

嵌入式命令系统里，错误信息本身就是调试能力的一部分。
语法错、参数错、状态错，最好能区分开。

5. 控制代码规模

MCU 项目要避免为了“讲模式完整”而引入过重的抽象。
很多时候：

- 命令树
- token 切分
- 参数解释
- 上下文执行

已经足够形成一套轻量解释器。

九、典型应用

下面这些都属于大量开源项目和厂家平台里的默认做法：

1. Zephyr Shell

Zephyr shell 默认就是命令树组织方式：根命令、静态子命令、动态子命令，再配合参数解析与执行。
它不是简单的字符串分发，而是一门轻量命令语言的解释过程。

2. ESP-IDF Console

ESP-IDF 的 console 组件提供命令注册、参数切分、参数解析和 REPL 组织能力。这本质上就是在解释一套交互式命令语言。

3. U-Boot 命令行

U-Boot 的命令行支持命令解析、环境变量、分号串接，甚至可以使用更强的 hush shell。这说明 bootloader 里经常不是单条命令分发，而是解释一门小型脚本语言。

4. 模组和设备的 AT 命令解析器

大量无线模组、通信模组和厂家调试口默认都使用 AT 风格语法：

- 命令字
- 参数
- 查询/设置/执行三种语义

这正是解释器模式最常见的工业落点之一。

总结

解释器模式在嵌入式 C 里的本质，就是把输入当成一门小语言来理解，而不是把它当成一个简单命令号来分派。